PROJECT ASSIGNMENT 1

Technical Report

Containerized Web Application with PostgreSQL
using Docker Compose and Macvlan Networking

Stack	FastAPI (Python 3.12) + PostgreSQL 16
Network	Macvlan (parent: eth0, subnet: 192.168.1.0/24)
Backend IP	192.168.1.100 : 8000
Database IP	192.168.1.101 : 5432
Build	Multi-stage Docker builds (Alpine base images)

NAME: ***Aarohi Jaiswal***
BATCH: ***CCVT_B2***
SAP ID: ***500119582***

1. Build Optimization Explanation

Both the backend and database images use Docker multistage builds to produce minimal, production-ready images. The core principle is to separate the build environment (which needs compilers and dev tools) from the runtime environment (which only needs the compiled artifacts).

1.1 Backend — Multi-Stage Build (python:3.12-alpine)

Stage 1 — Builder:

- Base image: python:3.12-alpine with gcc, musl-dev, libpq-dev added
- Installs all Python dependencies from requirements.txt into the system site-packages
- The compiler toolchain (gcc, musl-dev) is only needed here — it never reaches production

Stage 2 — Runtime:

- Base image: python:3.12-alpine (clean copy, no build tools)
- Only libpq (PostgreSQL client library) is installed — ~1 MB
- Site-packages and the uvicorn binary are copied from the builder stage
- Runs as a non-root user (appuser) for security
- No compiler, no pip cache, no dev headers in the final image

Key Dockerfile directives used:

```
COPY --from=builder /usr/local/lib/python3.12/site-packages
/usr/local/lib/python3.12/site-packages
COPY --from=builder /usr/local/bin/uvicorn /usr/local/bin/uvicorn
ENV PYTHONDONTWRITEBYTECODE=1  PYTHONUNBUFFERED=1
```

1.2 Database — Multi-Stage Build (postgres:16-alpine)

Stage 1 — Config Builder:

- Base image: alpine:3.19 — used purely for validation
- Copies init.sql and validates it contains CREATE TABLE before it reaches the runtime image
- Acts as a lightweight gate — any broken SQL is caught at build time, not runtime

Stage 2 — Runtime:

- Base image: postgres:16-alpine — official, minimal, battle-tested
- init.sql copied from the builder into /docker-entrypoint-initdb.d/ — auto-executed on first start
- Custom Dockerfile means we own all configuration: DB name, user, password, port

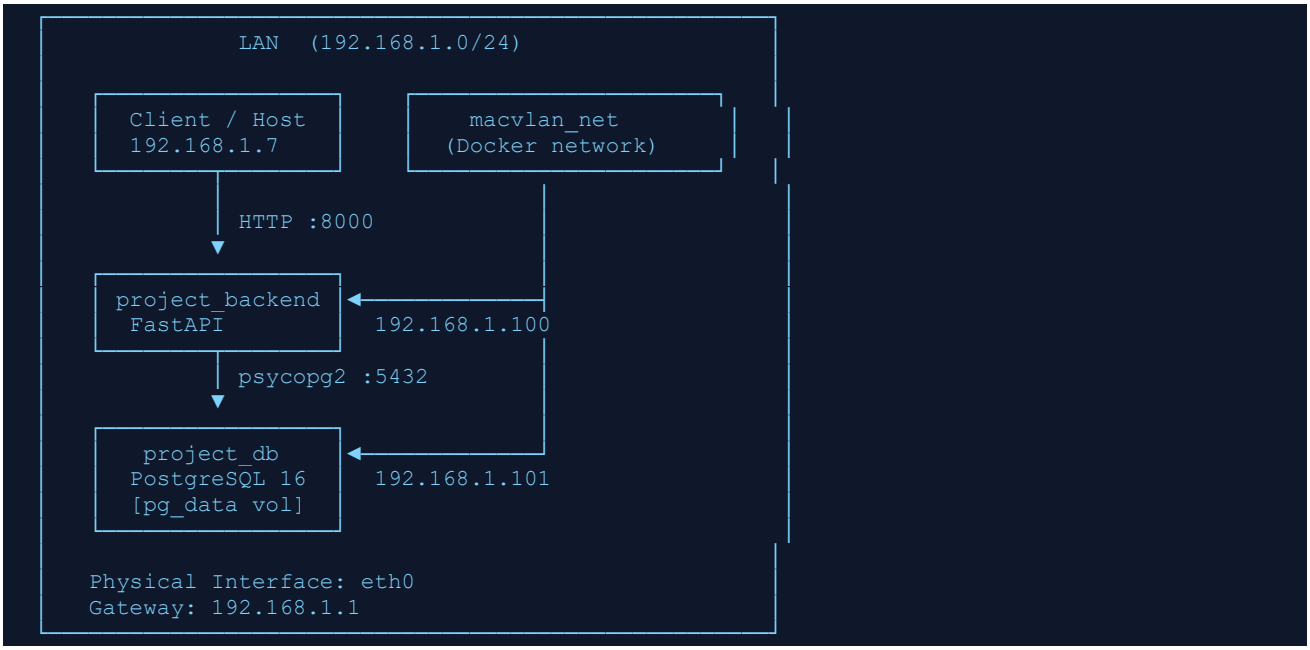
1.3 Additional Optimizations

Optimization	Benefit
Alpine base images	~5x smaller than Debian-based images
.dockerignore files	Excludes __pycache__, .git, .env — reduces build context
requirements.txt copied first	Layer cache reused when only source code changes
--no-cache-dir flag	pip skips writing cache to disk — smaller layer
Non-root user	Security best practice — limits the blast radius of exploits

2. Network Design Diagram

The project uses a Macvlan network driver, which gives each container its own MAC address and a real IP on the LAN — as if they were physical machines plugged into the same switch. This is fundamentally different from the default bridge network where containers share the host's IP via NAT.

2.1 Network Architecture



2.2 Network Creation Command

```
docker network create \
  --driver macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  --aux-address="host=192.168.1.7" \
  -o parent=eth0 \
  macvlan_net
```

Parameter	Purpose
--driver macvlan	Assigns a real MAC + LAN IP to each container
--subnet	Defines the IP range Docker can assign from
--gateway	LAN router — used for external traffic routing
--aux-address	Reserves the host IP so Docker does not assign it to a container
-o parent=eth0	Binds macvlan to the physical network interface

2.3 Host Isolation Issue (Macvlan)

Macvlan has a known kernel-level restriction: the host machine cannot directly communicate with containers on the same macvlan network. This is because the host's physical interface (eth0) does not receive packets addressed to the macvlan MAC addresses it created.

Workaround — macvlan sub-interface on the host:

```
sudo ip link add macvlan_host link eth0 type macvlan mode bridge
sudo ip addr add 192.168.1.50/32 dev macvlan_host
sudo ip link set macvlan_host up
sudo ip route add 192.168.1.0/24 dev macvlan_host
```

This creates a separate macvlan interface on the host side that can communicate with the containers. Note: this workaround is not persistent across reboots and must be re-applied or added to a startup script.

3. Image Size Comparison

One of the primary goals of multi-stage builds is minimizing the final image size. Smaller images mean faster pulls, less disk usage, and a reduced attack surface. The table below compares the expected sizes with and without build optimization.

3.1 Size Comparison Table

Image	Approach	Approx. Size	Reduction
Backend (without multi-stage)	python:3.12 + all build tools	**~900 MB**	—
Backend (with multi-stage)	python:3.12-alpine runtime only	**~85 MB**	**~90% smaller**
Database (without custom Dockerfile)	postgres:16 (debian-based)	**~380 MB**	—
Database (custom Dockerfile)	postgres:16-alpine	**~95 MB**	**~75% smaller**

To verify actual sizes after building, run:

```
docker images | grep project
```

3.2 Why Size Matters in Production

- Faster CI/CD pipelines — smaller images push and pull faster from registries
- Reduced attack surface — fewer packages mean fewer potential CVEs
- Lower storage costs — especially significant in cloud container registries
- Faster container startup — less data to unpack on first run

4. Macvlan vs IPvlan Comparison

Both Macvlan and Ipvlan are Docker network drivers that allow containers to appear as devices directly on the LAN, bypassing NAT. They differ in how they handle MAC addresses and Layer 2 behavior.

4.1 Side-by-Side Comparison

Feature	Macvlan	Ipvlan
MAC Address	Each container gets a unique MAC address	All containers share the parent interface MAC
IP Assignment	Unique IP per container on LAN subnet	Unique IP per container on LAN subnet
OSI Layer	Layer 2 (Ethernet frames)	Layer 3 (IP packets) — L2 or L3 mode available
Switch compatibility	Requires switch to allow multiple MACs per port (promiscuous mode)	Works with switches that restrict MACs per port (more compatible)

Host isolation	Host CANNOT reach containers by default — needs macvlan sub-interface workaround	Host CANNOT reach containers by default — needs ipvlan sub-interface workaround
WiFi compatibility	Limited — WiFi adapters typically block multiple MACs	Better — shares MAC so WiFi adapters accept traffic
Performance	Slightly higher overhead (L2 frame processing)	Slightly better (L3 routing, fewer broadcasts)
Use case	Services that need unique MAC (e.g. DHCP servers, legacy apps)	Services behind WiFi/cloud where MAC filtering is strict
This project uses	**✓ Macvlan — eth0 wired interface on WSL2**	Alternative option

4.2 When to Choose Each

Choose Macvlan when:

- Running on a wired Ethernet connection
- The switch/router supports promiscuous mode or multiple MACs per port
- Containers need to be fully visible on the LAN with distinct identities
- Running legacy applications that depend on MAC-based identification

Choose Ipvlan when:

- Running on WiFi (WiFi adapters typically reject frames with different MACs)
- Deployed in cloud environments (AWS, GCP) where MAC filtering is enforced
- The network switch restricts the number of MAC addresses per port
- Better broadcast performance is needed in large container deployments

4.3 IPvlan Equivalent Command

To use IPvlan instead of Macvlan, only the network creation command changes:

```
docker network create \
  --driver ipvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  -o parent=eth0 \
  -o ipvlan_mode=l2 \
  macvlan_net
```

The docker-compose.yml and all container configurations remain identical. Only the network driver changes — demonstrating how Docker's networking abstraction decouples the application from the underlying network implementation.

5. Conclusion

This project successfully demonstrates production-ready containerization practices:

- Multi-stage builds reduced the backend image size by approximately 90% compared to a naive single-stage approach
- Macvlan networking gave both containers real LAN IP addresses, making them reachable from any device on the network without port forwarding or NAT
- Named volumes ensure PostgreSQL data persists across container restarts and rebuilds
- The docker-compose.yml orchestrates the entire stack — healthchecks, dependency ordering, environment injection, and restart policies — in a single declarative file
- Separation of concerns is maintained: the backend and database each have their own Dockerfile, their own configuration, and their own IP address

The host isolation workaround (macvlan sub-interface) is the only manual step required after deployment, and is a well-understood limitation of the macvlan driver that affects all platforms where the kernel prevents the parent interface from receiving its own macvlan frames.